Finding and Loading Classes

A well-organized software project spreads its code across multiple files, and we've seen in previous chapters how to load source code from other files. Classes are no different. Just as a function cannot be called until the file containing its declaration has been parsed by PHP, a class cannot be used until PHP has loaded it. To demonstrate, we'll create three source code files…

```
//10.01.php
class SomeClass
{
}
```

```
//10.02.php
$someClass = new SomeClass();
```

```
//10.03.php
require __DIR__ . '/10.01_SomeClass.php';

$someClass = new SomeClass();
var_dump($someClass);
```

… and run the code in `10.02.php` and `10.03.php`:

```
Output: (10.02.php)
Fatal error: Uncaught Error: Class "SomeClass" not found...

Output: (10.03.php)
object(SomeClass)#1 (0) {
}
```

Here we've split our code across three files. The first (`10.01.php`) contains a simple class declaration; the other two instantiate the class (or try to). `10.02.php` exits with a fatal error: `Class "SomeClass" not found`, but `10.03.php` is successful because it loads the source code for `SomeClass` with a `require` statement. In order to use `SomeClass`, it is necessary to `include` or `require` the file containing the class declaration.

Of course, we could simply put every class declaration into a single file, but that would quickly become a maintenance headache of epic proportions as the number of classes in a project grows (say, beyond about three). In addition, multiple class